

A. Hadoop Storage File System Your first objective is to create a directory structure in HDFS using HDFS commands. Create the local files using Linux commands and move the files to HDFS directory and vice versa.

- i. Write a command to create the directory structure in HDFS.
- ii. Write a Command to move file from local unix/linux machine to HDFS.

B. Viewing Data Contents, Files and Directory Try to perform these simple steps:

- i. Write HDFS command to see the contents of files in HDFS.
- ii. Write HDFS command to see contents of files which are present in HDFS.

C. Getting Files data from the HDFS to Local Disk

- i. Write a HDFS command to copy the file from HDFS to local file system.

```
cd %HADOOP_HOME%\sbin
start-dfs.cmd
start-yarn.cmd
```

Step 1.1: Create folder

Run in CMD:

```
mkdir C:\data
```

♦ Step 1.2: Create file

Run:

```
notepad C:\data\input.txt
```

👉 It will ask:

Do you want to create a new file? → **Click YES**

♦ Step 1.3: Add content

Paste this:

```
hello hadoop hello world
hadoop is big data
```

👉 Save and close

♦ Step 1.4: VERIFY (very important)

Run:

dir C:\data

● Expected output:

You should see:

Input.txt

Step 2.1: Create correct directory

```
hdfs dfs -mkdir -p /user/hadoop/test
```

◆ **Step 2.2: Upload correctly**

```
hdfs dfs -put C:\data\input.txt /user/hadoop/test
```

◆ **Step 2.3: Verify**

```
hdfs dfs -ls /user/hadoop/test
```

STEP 3: View File Content (Now do this)

◆ **Command:**

```
hdfs dfs -cat /user/hadoop/test/input.txt
```

```
hdfs dfs -tail /user/hadoop/test/input.txt
```

● Expected Output

hello hadoop hello world

hadoop is big data

Copy with different name

```
hdfs dfs -get /user/hadoop/test/input.txt C:\data\input_copy.txt
```

◆ **Verify**

Step 1: Create Input File

- Create a text file with words to count.
- Example:

C:\data\week_8o.txt

Step 2: Start Hadoop Services

Open CMD and run:

```
start-dfs.cmd
```

```
start-yarn.cmd
```

Step 3: Upload File to HDFS

Run the following commands:

```
hdfs dfs -mkdir -p /user/hadoop/input
```

```
hdfs dfs -put C:\data\week_8o.txt /user/hadoop/input
```

```
hdfs dfs -ls /user/hadoop/input
```

(Use the last command to check if the file is uploaded successfully.)

Step 4: Create Java Program

- Create a folder in C drive:

C:\hadoop_work

- Inside it, create:

WordCount.java

- Paste the following code:

```
import java.io.IOException;
```

```
import java.util.StringTokenizer;
```

```
import org.apache.hadoop.conf.Configuration;
```

```
import org.apache.hadoop.fs.Path;
```

```
import org.apache.hadoop.io.IntWritable;
```

```
import org.apache.hadoop.io.Text;
```

```
import org.apache.hadoop.mapreduce.Job;
```

```
import org.apache.hadoop.mapreduce.Mapper;
```

```
import org.apache.hadoop.mapreduce.Reducer;
```

```
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
```

```
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```
public class WordCount {
```

```

public static class TokenizerMapper
extends Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        StringTokenizer itr = new StringTokenizer(value.toString());

        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}

public static class IntSumReducer
extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context)
        throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable val : values) {
            sum += val.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "word count");

```

```

        job.setJarByClass(WordCount.class);

        job.setMapperClass(TokenizerMapper.class);
        job.setReducerClass(IntSumReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Step 5: Compile and Create JAR

In CMD:

```
cd C:\hadoop_work
```

```
ren WordCount.java.txt WordCount.java
```

Compile:

```
javac -classpath
"%HADOOP_HOME%\share\hadoop\common\*; %HADOOP_HOME%\share\hadoop
\mapreduce\*; %HADOOP_HOME%\share\hadoop\hdfs\*" -d . WordCount.java

```

Create JAR:

```
jar -cvf wordcount.jar *
```

Step 6: Remove Old Output (if exists)

```
hdfs dfs -rm -r /user/hadoop/output
```

Step 7: Run the Program

```
hadoop jar wordcount.jar WordCount /user/hadoop/input /user/hadoop/output
```